

# 面向资源约束量子布尔函数综合的神经蒙特卡洛树搜索方法

中文论文稿 v7

2026 年 7 月 8 日

## 摘要

量子布尔函数 oracle 综合把经典谓词嵌入可逆量子线路，是 Grover 搜索、密码分析和若干量子算法子程序中的基础步骤。由于容错量子计算中非 Clifford 门代价高，逻辑层 T-count、CNOT、线路深度和辅助比特数会直接影响资源估计。本文将 oracle 综合建模为代数正规形 (algebraic normal form, ANF) 和固定极性 Reed-Muller (fixed-polarity Reed-Muller, FPRM) 项集合上的资源加权搜索问题，提出结合神经动作先验、蒙特卡洛树搜索、布尔环线性因子筛选、结构级 depth policy 和保守 skip guard 的 Resource-NMCTS 框架。该框架不依赖 SSHR 的 parallelotope cover 空间；SSHR 在本文中仅作为 CNOT-oriented small-scale baseline。实验显示，在  $n \leq 6$  的 177 个传统函数上，Pareto-Resource-NMCTS 相对 ESOP cube beam 获得 174/0/3 的 score 胜/负/平，平均 score 降低 36.09%；相对 weighted ESOP-MILP 获得 167/3/7，平均 score 降低 29.84%。在  $n = 18$  的 12 个随机 ANF 函数上，adaptive depth-2 Boolean-ring screen 相对单层筛选平均 score 降低 6.47%，完整 Resource-NMCTS 相对 adaptive screen 进一步降低 23.85%。在  $n = 20$  的 6 个边界函数上，adaptive depth-2 screen 相对单层筛选平均 score 降低 7.47%，screen-gated Resource-NMCTS 在保持资源不变的同时把平均运行时间从 77.10 s 降至 31.18 s。进一步的结构级 depth policy 在 held-out  $n = 20$  项集上相对单层和 depth-1 screen 分别降低 6.57% 和 2.57% 的平均 score；保守 depth-2 skip guard 在另一组 held-out  $n = 20$  项集上与固定 depth-2 screen 全部持平，其中 static-direct 模式在 96 个测试样本上相对固定 depth-2 和 all-depth adaptive 分别节省 0.54% 和 24.74% 的平均时间，shallow-staged 高置信模式把辅助 48 样本切片上的安全跳过覆盖率提高到 8/48。结果支持一条独立于 SSHR 的资源搜索论文主线，同时也给出边界：当前 learned prior 的高维独立质量收益仍弱，结构级 AI 仍需从小覆盖率安全门控推进到更大幅度的质量和时间收益。

**关键词：**量子 oracle 综合；布尔函数综合；神经蒙特卡洛树搜索；布尔环；ANF；FPRM；资源约束综合

## 1 引言

给定布尔函数  $f : \{0, 1\}^n \rightarrow \{0, 1\}$ ，量子 oracle 通常实现为

$$|x\rangle|y\rangle \mapsto |x\rangle|y \oplus f(x)\rangle. \quad (1)$$

这一变换看似只是经典谓词的可逆嵌入，但在实际量子算法中往往成为资源瓶颈。若 oracle 的非线性部分被逐项展开，T-count 和深度会随高次数项和项数迅速增长；若为了降低 T-count 大量复用中间结果，又可能增加 CNOT、辅助比特和调度深度。因此，oracle synthesis 不能只看某一种门数，而应作为一个多资源搜索问题处理。

ANF 是布尔函数 oracle 的自然输入表示。若

$$f(x) = \bigoplus_{m \in \mathcal{M}} \prod_{i \in m} x_i, \quad (2)$$

则每个单项式都可以被翻译为受控乘法结构，并异或到输出位。该路线透明且容易验证，但会忽略不同项之间共享的 AND 因子、线性奇偶结构和极性重写机会。已有 XAG、ROS、BDD 和 SSHR 等方法分别从 XOR/AND 图、资源约束 LUT 映射、决策图和 parallelotope cover 角度改进 oracle 或可逆逻辑综合 [2, 3, 4, 6]。这些工作说明中间表示会决定可暴露的资源结构，但它们并未直接回答本文关心的问题：能否把 ANF/FPRM 项集合本身作为搜索状态，并用 AI 方法在该状态空间中选择资源更优的因子分解？

本文的一句话论证是：在逻辑层量子布尔函数 oracle 综合中，ANF/FPRM 项集合上的资源感知神经搜索，配合布尔环线性因子筛选和结构级门控，可以在传统函数和  $n = 18, 20$  高维随机项集合上降低 T-count 与加权逻辑资源；当前边界是结论不覆盖硬件 mapping、routing、真实噪声模型，也不能声称 CNOT-only 全面优于 SSHR。

## 2 相关工作与定位

Xor-And-Inverter Graphs (XAG) 将 XOR 与 AND 结构分离，使 AND 节点数量与非 Clifford 资源直接相关。Meuli 等人将 XAG 用于 quantum compilation[2]，并从 multiplicative complexity 角度讨论低 T-count oracle 的来源 [1]。ROS 将 oracle synthesis 建模为 resource-constrained LUT mapping[3]。这些方法强调布尔中间表示的重要性；本文与它们的共同点是重视符号布尔结构，差异在于本文不以 XAG 或 LUT 为唯一中间表示，而是在 ANF/FPRM 项集合上直接搜索可复用因子。

BDD-based reversible synthesis 使用决策图处理较大布尔函数 [4]。Back-end-aware oracle synthesis 进一步把 XAG 优化与后端质量指标连接起来 [5]。本文目前只处理逻辑层资源估计，因此与后端感知工作互补而非替代。本文报告的 depth 是逻辑层线路深度，不包含设备连通性、routing、magic-state factory 调度和物理噪声。

SSHR 使用 spatial structures of parallelotopes 面向 small-scale Boolean functions 优化 CNOT 指标 [6]。本文保留 SSHR-H/SSHR-I 作为小规模 baseline，是因为它能暴露本文方法在 CNOT 指标上的 trade-off。但本文内部不使用 parallelotope candidate set，也不把目标写成“AI-SSHR”。更准确的定位是：本文把 Boolean oracle synthesis 重新表述为 ANF/FPRM 因子搜索，并在 T-count、CNOT、深度和辅助比特的加权资源口径下比较不同综合路线。

学习式搜索已用于多个离散综合任务。ShortCircuit 使用 AlphaZero 风格搜索生成经典布尔电路 [7]；Gumbel AlphaZero 被用于 Clifford+T unitary synthesis[8]；ZX-calculus 优化中出现了强化学习和图神经网络驱动的 rewrite-rule 选择 [9]；MonteQ 使用 MCTS 处理量子线路综合中的动作排序 [10]。这些工作支持学习式搜索适合组合综合空间，但它们的动作空间不是 Boolean oracle 的 ANF/FPRM 因子分解。本文的 AI 部分因此应被写成面向 Boolean oracle 结构的动作排序和结构选择，而不是泛化的“RL 生成量子线路”。

表 1: 本文锁定的术语与边界。

| 术语                  | 本文含义                                                                |
|---------------------|---------------------------------------------------------------------|
| Resource-NMCTS      | 在 ANF/FPRM 项集合上进行资源感知搜索的主方法，包括神经先验、MCTS、guard 和 portfolio 选择。       |
| Boolean-ring screen | 在 square-free Boolean ring 中搜索可复用线性因子的结构筛选分支，允许线性因子与 quotient 共享变量。 |
| Depth policy        | 预测 single、depth-1 或 depth-2 screen 的结构级神经策略。                        |
| Skip guard          | 保守判断是否跳过 depth-2 screen 的二分类门控；当前目标是 zero-loss，而非最大跳过率。             |
| SSHR                | CNOT-oriented small-scale baseline；本文方法不从 SSHR 搜索空间出发。              |

### 3 问题定义与资源模型

本文输入为布尔函数的 truth table 或 ANF 表示，输出为实现式 (1) 的逻辑层可逆线路。所有候选线路必须通过 truth-table 级语义验证。候选选择使用统一资源分数

$$\text{score} = T + 0.04 \text{ CNOT} + 0.015 \text{ depth} + 0.01 \text{ gates} + 2 \text{ ancilla}. \quad (3)$$

该分数用于实验排序，不代表特定硬件平台的真实物理成本。为避免单一指标掩盖 trade-off，本文同时报告 T-count、CNOT、depth、gate count 和 peak ancilla。

## 4 方法

### 4.1 状态、动作与搜索目标

Resource-NMCTS 的状态是当前尚未综合的项集合  $\mathcal{M}$ 。一次动作选择一个可计算、可反算的局部结构，将  $\mathcal{M}$  分解为 quotient 子问题和 rest 子问题，再递归生成线路。候选动作包括公共单项式因子、FPRM 极性重写后的公共项、CNOT-only 线性因子、布尔环线性因子，以及由神经模型扩展的 root actions。每个动作都必须能被翻译为 compute/action/uncompute 形式，保证最终线路语义可验证。

搜索目标不是最小化某一个符号复杂度，而是最小化式 (3) 下的逻辑资源。神经动作先验根据候选覆盖率、项数变化、平均次数、因子大小、即时资源收益和子问题规模等特征对动作排序。MCTS 在固定预算内探索候选组合，并由 baseline-preserving guard 兜底：如果学习候选不优于确定性 baseline，则最终返回 baseline 结果。这个设计使 AI 模块可以引入新候选，同时降低负迁移导致资源退化的风险。

### 4.2 布尔环线性因子

旧 linear-pair 分支主要搜索形如

$$(x_i \oplus x_j \oplus \dots) \cdot h(x) \quad (4)$$

的局部结构，并要求线性因子变量与 quotient  $h(x)$  变量不相交。该约束便于实现，但会漏掉 square-free Boolean ring 中合法的重叠变量结构。布尔环线性因子允许二者共享变量，并在展开时使用

$x_i^2 = x_i$  与 GF(2) 偶数项消去。例如

$$(x_i \oplus x_j)x_im = x_im \oplus x_ix_jm. \quad (5)$$

线路层面仍可先用 CNOT 计算线性奇偶量，再用该辅助量控制 quotient 子线路。因此，该扩展不是简单扩大 beam 宽度，而是把原先被实现约束排除的代数候选重新纳入可综合搜索空间。

### 4.3 结构级策略与门控

高维实验显示，固定 single screen、depth-1 screen 和 depth-2 screen 的收益依赖项集合结构。为使 AI 贡献从动作排序推进到结构选择，本文训练轻量 depth policy，输入项数、次数分布、变量覆盖密度、二元共现密度和 root Boolean-ring action 统计，输出应使用的 screen 深度。监督标签来自实际运行三种 screen 后按式 (3) 选择的最佳 depth。

由于固定 depth-2 screen 是很强的质量 baseline，本文又训练一个保守 depth-2 skip guard。Static-direct 模式只使用 ANF 项集合的静态结构特征，在预测 depth-1 足以与固定 depth-2 screen 持平时直接运行 depth-1，否则直接运行 depth-2。Shallow-staged 模式先运行 single/depth-1 screen，再用浅层观测特征判断是否跳过 depth-2。阈值在训练和验证切片上按 zero-false-skip 约束选择，shallow-staged 额外使用 0.95 高置信阈值下限。

另外， $n = 20$  边界切片中完整 Resource-NMCTS 的 Resource tail 与 adaptive screen 资源持平但运行时间更长。为控制该长尾，本文训练 screen-gate 判断 adaptive screen 是否已经足够、是否可以跳过昂贵 tail。该模块当前只作为运行时门控证据，不作为普适质量模型。

## 5 实验设置

实验分为四组。第一组使用  $n \leq 6$  的 177 个传统函数，与 direct ANF、AND-direct ANF、ESOP cube beam、ESOP-MILP、SSHR-H、ABC 和 BDD baseline 比较。第二组使用  $n = 18$  随机 ANF 函数，观察 adaptive screen 与完整 Resource-NMCTS 的差距。第三组使用  $n = 20$  边界函数，检验在 FPRM root beam 和 fast linear-pair 超时区域中，depth-2 screen 是否仍能给出可运行改进。第四组使用  $n = 14, 16, 18$  合成项集训练 depth policy 与 direct skip guard，并在 held-out  $n = 20$  项集上测试结构选择能力。

所有 paired comparison 只纳入函数名匹配、语义正确且未 timeout 的结果。出现 timeout 的方法保留为运行边界证据，但不参与正确结果的均值比较。本文目前所有实验均为逻辑层资源估计，不包含 hardware mapping 和 noise-aware scheduling。

## 6 结果

### 6.1 小规模传统函数

表 2 给出  $n \leq 6$  传统函数上的平均逻辑资源。相对 direct ANF，Pareto-Resource-NMCTS 平均 T-count 降低 72.25%，平均 score 降低 67.80%。相对 ESOP cube beam，Pareto-Resource-

表 2:  $n \leq 6$  传统函数切片上的平均逻辑资源。

| 方法                    | 平均 T        | 平均 CNOT     | 平均 depth    | 平均 ancilla | 平均 score    |
|-----------------------|-------------|-------------|-------------|------------|-------------|
| Direct ANF            | 184.1       | 134.2       | 134.6       | 1.33       | 194.3       |
| AND-direct ANF        | 101.0       | 166.5       | 166.9       | 2.18       | 115.2       |
| Resource-NMCTS        | 43.9        | 89.9        | 94.5        | 1.92       | 53.2        |
| Pareto-Resource-NMCTS | <b>40.4</b> | 83.0        | <b>88.0</b> | 2.03       | <b>49.6</b> |
| ESOP-MILP             | 83.6        | 133.5       | 159.6       | 2.32       | 96.7        |
| SSHR-H                | 81.0        | <b>67.6</b> | 88.7        | 1.36       | 88.2        |

表 3:  $n = 18$  随机 ANF 函数上的 adaptive screen 与完整 Resource-NMCTS。

| 方法                         | 平均 T           | 平均 CNOT         | 平均 depth        | 平均 score       | 平均时间 (s) |
|----------------------------|----------------|-----------------|-----------------|----------------|----------|
| Single Boolean-ring screen | 10389.33       | 16268.42        | 16268.50        | 11349.06       | 0.82     |
| Adaptive depth-2 screen    | 9767.00        | 15301.58        | 15301.67        | 10670.94       | 4.39     |
| Resource-NMCTS             | <b>6639.33</b> | <b>11380.92</b> | <b>11382.17</b> | <b>7315.62</b> | 226.18   |

NMCTS 获得 174/0/3 的 score 胜/负/平, 平均 score 降低 36.09%。相对 weighted ESOP-MILP, 获得 167/3/7, 平均 score 降低 29.84%。

该表也给出了本文主张边界: SSHR-H 的平均 CNOT 最低, 因此本文不能主张 CNOT-only 支配。更准确的表述是, 本文方法在当前资源分数下用一定 CNOT 与辅助比特 trade-off 换取更低 T-count 和更低 score。

## 6.2 $n = 18$ 随机项集合

表 3 显示, adaptive depth-2 Boolean-ring screen 相对单层 Boolean-ring screen 获得 11/0/1 的 score 胜/负/平, 平均 score 从 11349.06 降至 10670.94, 相对降低 6.47%。完整 Resource-NMCTS 平均 score 为 7315.62, 相对 adaptive screen 进一步降低 23.85%。这说明  $n = 18$  上快速 screen 是有效候选生成器, 但不能替代完整资源搜索。

## 6.3 $n = 20$ 边界切片

$n = 20$  切片更接近当前实现的扩展边界。FPRM root beam 和 fast linear-pair 在该切片中均出现 300 s task timeout。表 4 显示, adaptive depth-2 screen 平均 score 为 21331.24, 低于 single screen 的 22695.15 和 depth-1 screen 的 21988.52。相对 single screen, adaptive depth-2 screen 获得 5/0/1, 平均 score 降低 7.47%。集成该分支后, Resource-NMCTS 相对 AND-direct ANF 获得 5/0/1, 平均 score 降低 11.80%。

Screen-gated Resource-NMCTS 与完整 Resource-NMCTS 的 T-count、CNOT、depth、peak ancilla 和 score 全部持平, 但平均运行时间降低 61.31%。这不是新的质量收益, 而是长尾控制收益。它支持把门控机制纳入方法框架, 但不能说明 Resource tail 在所有高维函数上都可跳过, 因为  $n = 18$  中完整搜索仍明显强于 screen。

表 4:  $n = 20$  边界函数上的 screen 与 Resource-NMCTS 结果。

| 方法                          | T               | CNOT            | depth           | score           | 时间 (s) |
|-----------------------------|-----------------|-----------------|-----------------|-----------------|--------|
| Direct ANF                  | 42944.00        | 19600.33        | 19600.33        | 44037.63        | 10.36  |
| AND-direct ANF              | 21516.67        | 33635.67        | 33635.67        | 23491.15        | 10.41  |
| Single screen               | 20786.00        | 32492.50        | 32492.50        | 22695.15        | 10.70  |
| Depth-1 screen              | 20138.00        | 31480.50        | 31480.50        | 21988.52        | 12.03  |
| Adaptive depth-2 screen     | <b>19535.33</b> | <b>30542.50</b> | <b>30542.50</b> | <b>21331.24</b> | 20.67  |
| Resource-NMCTS              | <b>19535.33</b> | <b>30542.50</b> | <b>30542.50</b> | <b>21331.24</b> | 77.10  |
| Screen-gated Resource-NMCTS | <b>19535.33</b> | <b>30542.50</b> | <b>30542.50</b> | <b>21331.24</b> | 31.18  |

表 5: 结构级 depth policy 在 held-out  $n = 20$  项集上的比较。

| 比较                           | 函数数 | score 胜/负/平 | 平均 $\Delta$ score | 平均 $\Delta$ time |
|------------------------------|-----|-------------|-------------------|------------------|
| Policy vs single screen      | 48  | 42/0/6      | -6.57%            | +2224.00%        |
| Policy vs depth-1 screen     | 48  | 34/0/14     | -2.57%            | +257.27%         |
| Policy vs depth-2 screen     | 48  | 0/5/43      | +0.28%            | -10.85%          |
| Policy vs all-depth adaptive | 48  | 0/5/43      | +0.28%            | -34.71%          |

## 6.4 结构级 AI 策略

表 5 给出 held-out  $n = 20$  项集上的 depth policy 结果。策略网络在  $n = 14, 16, 18$  上训练, 在 48 个  $n = 20$  测试项集上评估。它相对 fixed single screen 获得 42/0/6, 平均 score 降低 6.57%; 相对 fixed depth-1 screen 获得 34/0/14, 平均 score 降低 2.57%。相对 all-depth adaptive 评估, policy 的平均 score 只高 0.28%, 但平均运行时间降低 34.71%。

固定 depth-2 screen 在 score 上仍略优。为去掉这一质量缺口, 本文训练了保守 depth-2 skip guard。表 6 显示, static-direct 模式在 held-out  $n = 20$  test 中没有 false skip, 96 个样本全部与固定 depth-2 screen score 持平, 并相对固定 depth-2 与 all-depth adaptive 分别节省 0.54% 和 24.74% 的平均时间。Shallow-staged 高置信模式在辅助 48 样本切片上把安全跳过覆盖率提高到 8/48, 但由于先运行 shallow screen, 相对固定 depth-2 本身仍更慢。

这个 guard 的意义是把结构级 policy 相对 fixed depth-2 的 score 缺口从 +0.28% 降为 0, 并把上一版 staged guard 相对 all-depth adaptive 的时间优势从 2.87% 提高到 24.74%。更重要的是, static-direct 不再比单独固定 depth-2 慢, 而是在 held-out test 上取得 0.54% 的小幅正收益。Shallow-staged 利用浅层观测提高 safe skip 覆盖率, 但仍有额外运行时开销。因此它应被解释为质量安全的 direct-dispatch 方向已经打通, 而不是最终高维质量模型已经完成。

## 7 讨论与局限

本文最稳妥的贡献表述有三点。第一, 方法主线独立于 SSHR。本文不是把 SSHR 改成 AI 版本, 而是在 ANF/FPRM 项集上做资源加权搜索; SSHR 的作用是 CNOT-oriented 小规模 baseline。第二, 布尔环线性因子提供了明确的结构扩展。它允许线性因子与 quotient 共享变量, 捕获旧实现约束排除的候选, 并在  $n = 18, 20$  上稳定改善单层 screen。第三, 结构级 AI 已有初步正向证据。Depth policy 学会了何时需要更深 screen, 保守 depth-2 guard 消除了相对 fixed depth-2 的 score



表 6: 保守 depth-2 skip guard 的 held-out  $n = 20$  zero-loss 模式比较。

| 模式             | 执行方式   | 样本数 | skip depth-2 | false skip | score 胜/负/平 | $\Delta t$ vs D2 | $\Delta t$ vs all-depth |
|----------------|--------|-----|--------------|------------|-------------|------------------|-------------------------|
| Static-direct  | direct | 96  | 4            | 0          | 0/0/96      | -0.54%           | -24.74%                 |
| Shallow-staged | staged | 48  | 8            | 0          | 0/0/48      | +29.10%          | -7.81%                  |

缺口并减少 all-depth adaptive overhead, screen-gate 也能在  $n = 20$  边界切片上减少完整 Resource tail 的运行时间。

当前证据还不能支撑“高维神经搜索已经充分解决”的强结论。其一,  $n = 18$  上完整 Resource-NMCTS 明显强于 adaptive screen, 说明中等高维仍需要更深搜索; 其二,  $n = 20$  上完整 Resource-NMCTS 与 adaptive screen 资源持平, 说明该切片的收益主要来自结构筛选而非 MCTS tail; 其三, static-direct 的安全跳过覆盖只有 4/96, shallow-staged 虽然达到 8/48 但仍需先运行浅层 screen。因而, 本文更合适的结论是: 结构级 AI 已经能在 zero-loss 约束下做出部分资源调度决策, 但还没有形成大覆盖率、强质量增益的高维策略模型。

下一步工作应围绕三个可量化目标展开。第一, 把 shallow-staged 的 8/48 安全跳过能力蒸馏回 direct guard: 在 held-out  $n = 20$  上继续保持不劣于 fixed depth-2 screen 的 score, 同时把 direct 模式相对固定 depth-2 的时间优势从 0.54% 提高到至少 5%。第二, 把结构级策略扩展为分支选择器: 不仅选择 screen 深度, 还预测是否进入 FPRM polarity search、完整 Resource-NMCTS 或 screen-gated tail。第三, 补充函数级线路案例和外部 reversible-toolchain 对比, 使论文从表格结果走向可解释的综合机制。

## 8 结论

本文提出一种面向资源约束量子布尔函数综合的神经蒙特卡洛树搜索方法。该方法以 ANF/FPRM 项集合为状态, 以逻辑资源 score 为选择准则, 用布尔环线性因子、神经先验、MCTS、baseline guard、depth policy 和 screen-gate 共同生成候选线路。实验表明, Pareto-Resource-NMCTS 在  $n \leq 6$  传统函数上显著降低 T-count 与 score; adaptive depth-2 screen 在  $n = 18, 20$  高维随机函数上稳定改善单层 screen; screen-gate 在  $n = 20$  边界函数上保持资源不变并减少运行时间; 保守 skip guard 在 held-out  $n = 20$  上实现不劣于 fixed depth-2 的结构级 direct dispatch, 并相对 all-depth adaptive 减少 24.74% 平均时间。当前结果已经形成一条独立于 SSHR 的论文主线, 但要达到更强投稿标准, 还需要把结构级 AI 从小幅安全门控推进到更大覆盖率的质量收益。

## 参考文献

- [1] G. Meuli, M. Soeken, E. Campbell, M. Roetteler, and G. De Micheli. The role of multiplicative complexity in compiling low T-count oracle circuits. In *Proceedings of ICCAD*, 2019.
- [2] G. Meuli, M. Soeken, and G. De Micheli. Xor-And-Inverter Graphs for quantum compilation. *npj Quantum Information*, 8:7, 2022.

- [3] G. Meuli, M. Soeken, M. Roetteler, and G. De Micheli. ROS: Resource-constrained oracle synthesis for quantum computers. *Electronic Proceedings in Theoretical Computer Science*, 318:119–130, 2020.
- [4] R. Wille and R. Drechsler. BDD-based synthesis of reversible logic for large functions. In *Proceedings of DAC*, pp. 270–275, 2009.
- [5] M. Yu, A. Tempia Calvino, M. Soeken, and G. De Micheli. Back-end-aware fault-tolerant quantum oracle synthesis. In *Proceedings of ASP-DAC*, pp. 930–937, 2025.
- [6] Q. Zheng, Y. Xu, J. Zhang, Z. Su, and S. Zheng. CNOT oriented synthesis for small-scale Boolean functions using spatial structures of parallelotopes. In *Proceedings of ICCAD*, 2025.
- [7] D. Tsaras, A. Grosnit, L. Chen, Z. Xie, H. Bou-Ammar, and M. Yuan. ShortCircuit: AlphaZero-driven synthesis of Boolean circuits. *arXiv:2408.09858*, 2024.
- [8] S. Rietsch, A. Y. Dubey, C. Ufrecht, M. Periyasamy, A. Plinge, C. Mutschler, and D. D. Scherer. Unitary synthesis of Clifford+T circuits with reinforcement learning. In *IEEE International Conference on Quantum Computing and Engineering*, pp. 824–835, 2024.
- [9] J. Riu, J. Nogu  , G. Vilaplana, A. Garcia-Saez, and M. P. Estarellas. Reinforcement learning based quantum circuit optimization via ZX-calculus. *Quantum*, 9:1758, 2025.
- [10] M. Machiya, M. Menickelly, P. Hovland, and J. Liu. MonteQ: A Monte Carlo tree search based quantum circuit synthesis framework. *arXiv:2604.19029*, 2026.